Introduction to Python

Python is a high-level, interpreted, general-purpose programming language widely used in **cybersecurity, data analysis, artificial intelligence, automation, and web development**.

In cybersecurity, Python is particularly useful for:

- Network scanning and analysis
- Log analysis
- Malware analysis
- Penetration testing
- Security automation
- Cryptography
- Digital forensics
- Threat detection

---

1. Basic Python Syntax

Python syntax is designed to be simple and readable. Unlike languages such as C or Java, Python uses **indentation** to define blocks of code.

A simple Python program

```
print("Hello, Cybersecurity!")
```

Output:

```
Hello, Cybersecurity!
```

Comments

Comments are ignored by the Python interpreter and are used to explain code.

```
# This is a single-line comment

print("Security Analysis")
```

Python does not use { } to define code blocks.

---

2. Indentation

Indentation is an essential part of Python syntax.

```
age = 25

if age >= 18:
    print("Access granted")
```

The indented statement belongs to the if block.

Incorrect indentation can produce an error:

```
if age >= 18:
print("Access granted")
```

---

3. Variables

A variable stores a value.

```
username = "admin"

age = 30

score = 95.5
```

Python automatically determines the data type.

```
username = "admin"   # string
```

```
age = 30          # integer
score = 95.5      # float
active = True     # boolean
```

You can check the type using type():

```
print(type(username))
print(type(age))
print(type(score))
print(type(active))
```

---

4. Python Data Types

The most important built-in data types are:

Data Type	Description	Example
int	Integer numbers	100
float	Decimal numbers	10.5
str	Text	"admin"
bool	True/False	True
list	Ordered, mutable collection	[1, 2, 3]
tuple	Ordered, immutable collection	(1, 2, 3)
set	Unordered collection of unique values	{1, 2, 3}
dict	Key-value pairs	{"user": "admin"}
NoneType	Represents no value	None

---

5. Integer (int)

Integers are whole numbers.

port = 443

attempts = 5

Arithmetic operations:

a = 10

b = 3

print(a + b)

print(a - b)

print(a * b)

print(a / b)

print(a // b)

print(a % b)

print(a ** b)

In cybersecurity, integers can represent:

port = 22

failed_attempts = 5

packet_size = 1500

---

6. Floating-Point (float)

A float represents a number containing a decimal point.

risk_score = 0.85

network_speed = 100.5

Example:

temperature = 36.7

print(type(temperature))

Output:

<class 'float'>

---

7. String (str)

Strings represent text.

username = "admin"

message = "Unauthorized access detected"

Strings can use single or double quotation marks:

name = 'Python'

name = "Python"

String operations

username = "admin"

print(username.upper())

print(username.lower())

print(len(username))

Output:

ADMIN

admin

5

String indexing

word = "Python"

print(word[0])

print(word[1])

Output:

P

y

Python indexing starts at **0**.

---

8. Boolean (bool)

Boolean values represent two states:

True

False

Example:

authenticated = True

if authenticated:

    print("User authenticated")

Boolean values are extremely important in security decisions:

access_granted = True

malicious = False

---

9. Lists

A list stores multiple values and can be modified.

ports = [22, 80, 443, 8080]

Accessing an element:

print(ports[0])

print(ports[2])

Adding an element:

ports.append(3306)

Removing an element:

ports.remove(80)

Cybersecurity example:

```
suspicious_ips = [
    "192.168.1.10",
    "192.168.1.25",
    "10.0.0.15"
]
```

---

10. Tuples

A tuple is similar to a list, but its contents cannot normally be changed after creation.

coordinates = (192, 168, 1, 10)

Example:

protocol = ("TCP", 443)

print(protocol[0])

print(protocol[1])

Tuples are useful for representing fixed collections of related values.

---

11. Sets

A set contains **unique values**.

```
ports = {22, 80, 443, 80, 22}

print(ports)
```

Duplicate values are removed.

This is useful in cybersecurity when analyzing unique IP addresses, ports, domains, or hashes.

```
unique_ips = {
    "192.168.1.10",
    "192.168.1.10",
    "192.168.1.20"
}
```

---

12. Dictionaries

A dictionary stores information as **key-value pairs**.

```
user = {
    "username": "admin",
    "role": "administrator",
    "active": True
}
```

Accessing values:

```
print(user["username"])

print(user["role"])
```

Dictionaries are extremely important in cybersecurity programming because security events often contain structured information.

Example:

```
event = {
    "source_ip": "192.168.1.50",
    "destination_port": 443,
    "protocol": "TCP",
    "malicious": False
}
```

---

13. None

None represents the absence of a value.

```
result = None
```

Example:

```
password = None

if password is None:
    print("Password not provided")
```

---

14. Type Conversion

Python allows conversion between compatible data types.

```
port = "443"
```

port_number = int(port)

print(port_number)

print(type(port_number))

Other examples:

x = int("100")

y = float("10.5")

z = str(123)

Boolean conversion:

value = bool(1)

print(value)

---

15. Input from the User

Python uses input() to receive data.

username = input("Enter username: ")

print("Welcome", username)

Important: input() normally returns a **string**.

Therefore:

age = int(input("Enter your age: "))

---

16. Basic Operators

Arithmetic operators

+ Addition

- Subtraction

* Multiplication

/ Division

// Floor division

% Modulus

** Power

Comparison operators

== Equal

!= Not equal

> Greater than

< Less than

>= Greater than or equal

<= Less than or equal

Example:

```
risk = 0.8
if risk >= 0.7:
    print("High-risk request")
```

Logical operators

and

or

not

Example:

```
authenticated = True
high_risk = False
```

```
if authenticated and not high_risk:
    print("Access granted")
```

---

17. Conditional Statements

The basic structure is:

```
if condition:
    statement
elif another_condition:
    statement
else:
    statement
```

Cybersecurity example:

```
risk_score = 0.85
if risk_score >= 0.80:
    print("Access denied")
elif risk_score >= 0.50:
    print("Multi-factor authentication required")
else:
    print("Access granted")
```

This provides a simple foundation for implementing **risk-aware access-control logic**.

---

18. Basic Loops

for loop

```
ports = [22, 80, 443]

for port in ports:
    print(port)
```

while loop

```
attempts = 0

while attempts < 3:
    print("Authentication attempt")
    attempts += 1
```

Loops are commonly used for processing:

- IP addresses
- Network ports
- Log records
- Files
- Security events

---

19. Practical Cybersecurity Example

The following small program demonstrates variables, data types, conditions, and user input:

```
username = input("Username: ")

failed_attempts = int(input("Failed login attempts: "))
```

```
if failed_attempts >= 5:
    print("ALERT: Account may be under attack.")
else:
    print("Normal authentication activity.")
print("User:", username)
```